



**I302 - Aprendizaje Automático
y Aprendizaje Profundo**

Trabajo Práctico 3:

Redes Neuronales

Martín Bianchi

11 de mayo de 2025

Ingeniería en Inteligencia Artificial

Resumen

En este trabajo se abordó el problema de clasificación multiclase de caracteres japoneses utilizando redes neuronales artificiales. Se desarrollaron modelos desde cero con NumPy (M0, M1) aplicando diversas técnicas como regularización L2, dropout, early stopping, mini-batch SGD y Adam, logrando mejoras significativas en generalización. Luego, se replicó y extendió la arquitectura óptima en PyTorch (M2, M3, M4), permitiendo explorar variantes más profundas y analizar fenómenos de sobreajuste. El modelo final logró una alta precisión sobre el conjunto de test, y se utilizó para predecir distribuciones de probabilidad en un conjunto adicional de imágenes.

1. Introducción

En este trabajo se abordó el problema de clasificación multiclase de imágenes de caracteres japoneses utilizando redes neuronales artificiales, con un enfoque tanto teórico como práctico orientado al análisis de cómo distintas modificaciones arquitectónicas y técnicas de entrenamiento afectan el rendimiento y la capacidad de generalización de los modelos. El conjunto de datos utilizado consiste en imágenes en escala de grises de 28×28 píxeles, cada una representando un carácter japonés perteneciente a una de las 49 clases posibles. Por lo tanto, la entrada del algoritmo es una matriz de dimensión 28×28 (784 características planas) con valores normalizados entre 0 y 1, mientras que la salida esperada es una distribución de probabilidad sobre 49 clases, representando la probabilidad a posteriori de que una imagen pertenezca a cada clase.

Como primera instancia, se implementó desde cero una red neuronal multicapa utilizando únicamente NumPy, sin emplear frameworks de *machine learning* de alto nivel como PyTorch o TensorFlow. Esta red (modelo M0) cuenta con dos capas ocultas de 100 y 80 neuronas respectivamente, empleando la función de activación ReLU en las capas ocultas y softmax en la capa de salida. El entrenamiento se realizó mediante descenso por gradiente y *backpropagation*, adaptado para la clasificación multiclase con función de pérdida de entropía cruzada. Se evaluó la performance en términos de *accuracy*, *cross-entropy loss* y matriz de confusión, tanto en el conjunto de entrenamiento como en el de validación.

Posteriormente, se introdujeron mejoras progresivas al algoritmo de entrenamiento con el objetivo de evaluar de forma individual el impacto de cada técnica. Entre las optimizaciones aplicadas se incluyen *learning rate scheduling* (lineal y exponencial), *mini-batch stochastic gradient descent*, el uso del optimizador Adam, regularización mediante L2, *early stopping*, y *dropout*. Estas técnicas permitieron reducir el error de validación y acelerar la convergencia del entrenamiento. A partir de estas optimizaciones y una búsqueda experimental de hiperparámetros y arquitectura, se construyó el modelo M1, la mejor red neuronal obtenida usando implementación propia sin librerías externas.

A continuación, se trasladaron los conocimientos y configuraciones óptimas obtenidas al entorno PyTorch, entrenando un nuevo modelo (M2) que replicó exactamente la arquitectura e hiperparámetros del modelo M1, lo que permitió validar la consistencia entre implementaciones y comparar tiempos de entrenamiento. Luego se exploraron nuevas variantes de arquitectura y parámetros usando PyTorch para obtener un modelo aún más eficaz (M3), evaluando distintas combinaciones de capas ocultas y unidades. Finalmente, se

diseñó un modelo con sobreajuste explícito (M4), utilizando una arquitectura intencionalmente compleja sin regularización, con el objetivo de evidenciar la diferencia entre buen rendimiento en entrenamiento y pobre generalización en test.

Como cierre, se compararon sistemáticamente los cinco modelos desarrollados (M0, M1, M2, M3 y M4) sobre el conjunto de test, analizando diferencias en precisión, pérdida, y comportamiento general. Finalmente, utilizando el modelo con mejor desempeño, se resolvió un desafío adicional: predecir las probabilidades a posteriori para un conjunto adicional de imágenes (`X_COMP.npy`), generando un archivo CSV con la distribución de probabilidad de pertenencia a cada clase para cada muestra.

2. Métodos

Algoritmos y técnicas utilizadas

En este trabajo se desarrollaron desde cero modelos de redes neuronales feedforward para clasificación multiclase, con soporte para múltiples capas ocultas y diferentes técnicas de optimización y regularización. Las redes se entrenaron mediante **backpropagation**, usando la regla de la cadena para calcular los gradientes de la función de costo con respecto a cada uno de los parámetros de la red. La función de activación utilizada en las capas ocultas fue la función **ReLU**:

$$\text{ReLU}(z) = \max(0, z)$$

y en la capa de salida se utilizó la función **softmax**, que transforma los valores de activación en probabilidades:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

donde K es el número de clases. Para el entrenamiento, se empleó como función de costo la **entropía cruzada**:

$$\mathcal{L}(y, \hat{y}) = - \sum_{i=1}^K y_i \log(\hat{y}_i)$$

donde y_i representa el valor real (codificado one-hot) y $\hat{y}_i$ la probabilidad predicha para la clase i .

El modelo inicial, denominado **M0**, consistió en una red de dos capas ocultas con 100 y 80 unidades respectivamente, y se entrenó utilizando **descenso por gradiente estándar (GD)**. Luego se implementaron múltiples mejoras sobre esta base, evaluando individualmente su impacto en el rendimiento del modelo:

- **Rate scheduling**: se aplicaron esquemas de tasa de aprendizaje dinámicos, tanto lineales como exponenciales.
- **Mini-batch stochastic gradient descent (SGD)**: reemplazando el descenso por gradiente estándar por uno más eficiente y menos sensible al ruido.
- **ADAM**: optimizador adaptativo que ajusta la tasa de aprendizaje de cada parámetro individual-

mente en función del primer y segundo momento de los gradientes.

- **Regularización L2:** se añadió una penalización de la forma $\lambda \sum ||W||^2$ al costo, para reducir el overfitting.
- **Dropout:** durante el entrenamiento, se desactivó aleatoriamente una fracción de las neuronas en capas intermedias para prevenir coadaptaciones y mejorar la generalización.
- **Early stopping:** para detener el entrenamiento automáticamente cuando la pérdida de validación dejaba de mejorar.

Cada optimización se evaluó de forma aislada, manteniendo constante la arquitectura del modelo y los demás hiperparámetros, para aislar su efecto específico.

Conjunto de datos y preprocesamiento

Los datos fueron cargados desde archivos en formato `.npy`, consistentes en imágenes en escala de grises de tamaño 28×28 . Tras una visualización inicial para validar su correcta carga, los vectores planos fueron reestructurados a matrices bidimensionales para su inspección.

Posteriormente, el conjunto de datos se dividió de la siguiente manera:

- **Entrenamiento:** 60 %
- **Validación:** 20 %
- **Test:** 20 %

Todas las imágenes fueron **normalizadas** dividiendo por 255, de modo que los valores de píxeles quedaron en el rango $[0, 1]$. Las etiquetas fueron codificadas mediante **one-hot encoding**, en vista de que la función softmax requiere salidas probabilísticas multiclase.

Hiperparámetros y validación

Se exploraron distintas combinaciones de hiperparámetros según el optimizador utilizado. En todos los casos, se utilizó una regularización L2 pequeña ($\lambda = 0,01$) para reducir el overfitting y limitar la cantidad de combinaciones a evaluar. La búsqueda se estructuró del siguiente modo:

- **Optimización (solver):** GD, SGD, Adam
- **Learning rate:**
 - GD: 1.0
 - SGD: 0.01
 - Adam: 0.001
- **Tamaño de batch:** SGD, Adam: 64 y 128
- **Dropout:** 0 y 0.2

■ **Arquitectura:**

- Una a tres capas ocultas
- Ejemplos: [256], [100, 80], [256, 128], [128, 64, 32]

La selección de hiperparámetros se realizó mediante **validación cruzada hold-out**, utilizando el conjunto de validación reservado (20%). No se utilizó k -fold debido a las exigencias computacionales del entrenamiento completo de múltiples modelos.

El mejor modelo de nuestra implementación, **M1**, correspondió a una red con arquitectura **256-128**, con una tasa de aprendizaje de 1, regularización L2 de 0.01, sin dropout, y entrenado mediante **descenso por gradiente** estándar. La evaluación sobre el conjunto de validación mostró mejoras significativas tanto en loss como en accuracy frente a la versión base M0.

A continuación, replicamos dicha configuración en **PyTorch** (modelo **M2**) para aprovechar su infraestructura y validamos que los resultados fueran consistentes. Con esta herramienta, se exploraron nuevas arquitecturas (**M3**) variando la cantidad de capas ocultas y neuronas por capa. Se evaluaron configuraciones que incluían desde una sola capa de 64, 128, 256, 512 o 1024 neuronas, hasta arquitecturas más profundas como dos capas de 128 y 64 neuronas; 256 y 128 neuronas; tres capas de 128, 64 y 32 neuronas; 512, 256 y 128 neuronas; y arquitecturas de cuatro capas con 512, 256, 128 y 64 neuronas respectivamente.

Se determinó que el modelo con una única capa de 1024 unidades ofrecía la mejor generalización. Posteriormente, se construyó el modelo **M4** con una arquitectura de cuatro capas: **512-256-128-64**, con el objetivo explícito de observar fenómenos de *overfitting*, lo cual se confirmó a través del aumento en la brecha entre el accuracy de entrenamiento y validación.

Métricas de evaluación

Para evaluar el rendimiento de los modelos en todas las etapas, se utilizaron las siguientes métricas:

- **Accuracy:** proporción de predicciones correctas sobre el total de ejemplos.
- **Cross-entropy loss:** pérdida utilizada durante el entrenamiento, que mide la distancia entre las distribuciones reales y predichas.
- **Matriz de confusión:** permite analizar detalladamente los errores cometidos por clase.

Estas métricas se calcularon sobre los conjuntos de **entrenamiento**, **validación** y finalmente sobre el conjunto de **prueba** para los modelos M0, M1, M2, M3 y M4. Las comparaciones finales muestran el compromiso entre complejidad de arquitectura, capacidad de generalización, y tiempo de entrenamiento. El análisis detallado se complementa con tablas y gráficos que resumen la evolución de las métricas a lo largo del entrenamiento.

3. Análisis de Resultados

A lo largo del trabajo se entrenaron múltiples modelos con distintas configuraciones y técnicas de entrenamiento. En esta sección analizamos en profundidad los resultados más relevantes, evaluando rendimiento, capacidad de generalización y comportamiento frente al sobreajuste.

3.1. Modelo Base (M0)

El modelo M0 fue entrenado con descenso por gradiente estándar, sin técnicas adicionales de regularización ni ajuste dinámico de hiperparámetros. Como se observa en la Figura 1, la pérdida sobre el conjunto de entrenamiento continuó disminuyendo de forma sostenida a lo largo de las 500 épocas. Sin embargo, la pérdida en validación comenzó a aumentar alrededor de la época 200.

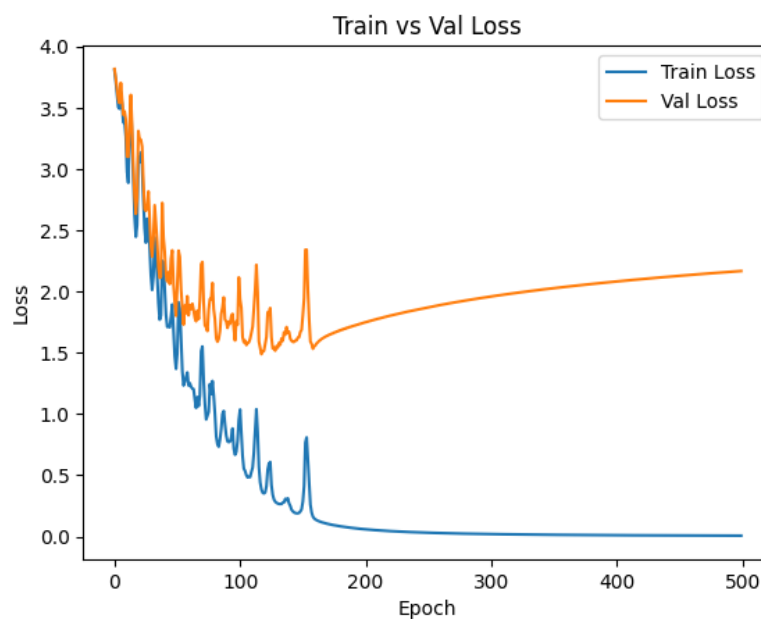


Figura 1: Evolución del loss en entrenamiento y validación para el modelo M0.

Este comportamiento evidencia un claro caso de *overfitting*, donde el modelo memoriza el conjunto de entrenamiento, pero pierde capacidad de generalización. Las métricas lo confirman: en entrenamiento se alcanza una **accuracy de 1.00** con **loss casi nulo**, mientras que en validación la accuracy cae a 0.62 y la pérdida sube a 2.17.

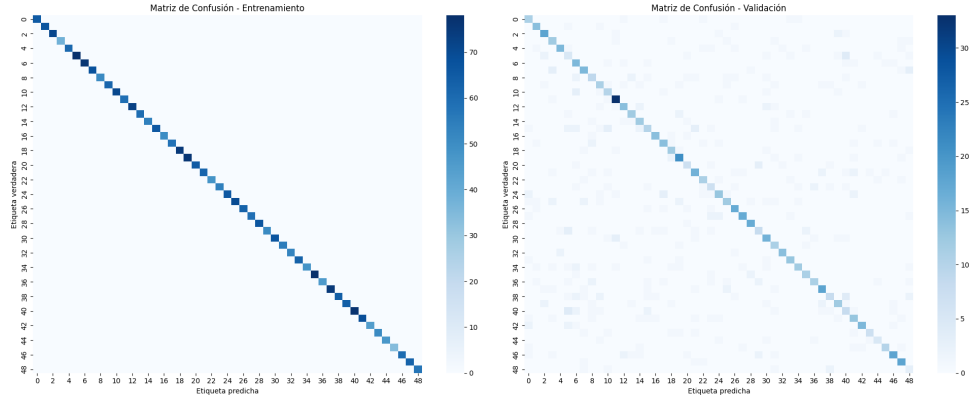


Figura 2: Matrices de confusión de M0 en entrenamiento (izquierda) y validación (derecha).

La Figura 2 refuerza esta observación: la matriz de entrenamiento muestra una diagonal perfecta, indicando que el modelo aprendió a clasificar todos los ejemplos vistos. En contraste, la matriz de validación presenta numerosos errores, evidenciando su incapacidad para generalizar a datos nuevos.

3.2. Técnicas de Mejora sobre M0

A fin de mitigar el sobreajuste observado en el modelo base **M0**, se aplicaron diversas técnicas de optimización y regularización. La Figura 3 ilustra el impacto de cada una sobre las métricas de entrenamiento y validación.

Resumen de Resultados					
	Tiempo (s)	Acc Train	Acc Val	Loss Train	Loss Val
m0	56.75	1.00	0.64	0.01	2.13
adam	16.85	0.87	0.60	0.60	1.70
schedule_exp	27.91	0.67	0.55	1.35	1.84
schedule_lin	22.50	0.93	0.60	0.36	1.57
sgd	18.42	0.87	0.58	0.63	1.73
L2	14.96	0.95	0.61	0.25	1.57
dropout	16.25	0.89	0.61	0.51	1.58

Figura 3: Evaluación del efecto de distintas técnicas de mejora sobre el modelo base M0: early stopping, rate scheduling, optimizadores y regularización.

Como puede observarse en la Figura 3, la técnica de **early stopping** permitió reducir notablemente el tiempo de entrenamiento sin afectar la accuracy ni incrementar la pérdida de validación. Esto la convierte

en una herramienta útil para evitar entrenamientos prolongados innecesarios, deteniendo el proceso una vez alcanzado el mejor punto de validación.

Respecto a los esquemas de **rate scheduling**, los resultados muestran un comportamiento dispar: el **esquema lineal** logró mantener una alta accuracy al tiempo que redujo de forma efectiva la pérdida, destacándose como una mejora real respecto al modelo base. Por el contrario, el **esquema exponencial** no solo aumentó el tiempo de entrenamiento sino que también empeoró la accuracy y el loss, resultando contraproducente. Esto puede deberse a que, como el modelo base ya aprende bien, un decay agresivo como el exponencial “apaga” el aprendizaje demasiado pronto. En cambio, el esquema lineal reduce el rate de forma más gradual, permitiendo ajustes finos en las últimas épocas sin cortar el progreso.

En cuanto a los **optimizadores**, tanto **Adam** como **SGD** mostraron una convergencia más rápida que el descenso por gradiente estándar. Sin embargo, no lograron superar en accuracy al modelo base. Esto podría estar relacionado con el hecho de que, al ser un dataset pequeño y poder usar todo el conjunto en cada paso, el descenso clásico funciona de forma estable y efectiva, mientras que los otros métodos, diseñados para minibatches, podrían no aportar ventajas claras en este caso.

Finalmente, las técnicas de **regularización** como **L2** y **dropout** resultaron efectivas para disminuir el sobreajuste. Ambas lograron reducir la pérdida de validación respecto a M0, con una penalización mínima en accuracy.

En conjunto, estos resultados permiten identificar cuáles estrategias aportan mejoras reales al modelo, y cuáles no justifican su costo computacional o complejidad adicional.

3.3. Mejor Configuración (M1)

La mejor combinación resultó ser una red con arquitectura **256-128**, **descenso por gradiente**, **regularización L2 (0.01)** y sin dropout. Esta configuración alcanzó un buen balance entre precisión y generalización:

- Entrenamiento: Accuracy = 0.96, Loss = 0.24
- Validación: Accuracy = 0.64, Loss = 1.47

Este modelo supera claramente al M0 en generalización. Llama la atención que un esquema simple como el descenso por gradiente estándar, combinado solo con L2, haya rendido mejor que variantes más sofisticadas. Esto sugiere que el dataset, aunque presenta cierta complejidad, no es lo suficientemente grande o ruidoso como para requerir mecanismos adaptativos. Además, al usar el conjunto completo en cada paso (batch gradient descent), se amortiguan fluctuaciones que otros optimizadores podrían sobreajustar. En este contexto, un modelo más simple evita sobrecomplicar la dinámica del entrenamiento y se adapta mejor a la estructura real de los datos.

3.4. Implementación en PyTorch (M2)

Se replicó la configuración M1 en PyTorch para evaluar si la implementación afectaba el desempeño. Los resultados fueron similares, aunque con una leve caída en las métricas:

- Entrenamiento: Accuracy = 0.81, Loss = 0.84
- Validación: Accuracy = 0.59, Loss = 1.52

Esto refuerza la solidez de la configuración elegida y sugiere que el cambio de framework no introduce mejoras automáticas. PyTorch ofrece ventajas principalmente al escalar a arquitecturas más complejas o al incorporar técnicas avanzadas, pero en este caso no resultó determinante.

3.5. Exploración Arquitectónica (M3)

Se evaluaron distintas configuraciones arquitectónicas variando el número y tamaño de capas ocultas. El mejor resultado se obtuvo con una red de **una sola capa oculta de 1024 unidades**, alcanzando:

- Entrenamiento: Accuracy = 0.86, Loss = 0.82
- Validación: Accuracy = 0.61, Loss = 1.53

Este desempeño, aunque competitivo, no representa una mejora significativa respecto a M1 o M2. Esto sugiere que el **dataset ya está siendo bien capturado por arquitecturas más simples**, y que agregar más neuronas no aporta una ganancia real.

3.6. Forzando Overfitting (M4)

El modelo M4 se diseñó para observar explícitamente el fenómeno de sobreajuste, siendo la mejor arquitectura para esto una profunda con 4 capas ocultas de tamaño decreciente: **[512, 256, 128, 64]**. Como se esperaba, el modelo alcanzó una performance perfecta en entrenamiento pero con una fuerte degradación en validación:

- Entrenamiento: Accuracy = 1.00, Loss = 0.001
- Validación: Accuracy = 0.62, Loss = 2.87

Este comportamiento confirma que una capacidad excesiva, sin regularización adicional, conduce a memorizar el set de entrenamiento en lugar de generalizar. Además, resulta interesante contrastar este modelo con el M0 original (NumPy, sin regularización), ya que ambos sufren de sobreajuste pero desde extremos distintos: M0 por falta de control, y M4 por exceso de complejidad. Esto evidencia que la solución óptima no siempre implica agregar más capas, sino encontrar un equilibrio adecuado entre capacidad y regularización.

3.7. Evaluación Final y Predicción

En esta etapa se compararon todos los modelos desarrollados (M0 a M4) con sus respectivos resultados sobre el conjunto de test. La Figura 4 muestra las métricas clave de cada modelo: accuracy y loss, mientras

que la Figura 5 presenta las matrices de confusión correspondientes.

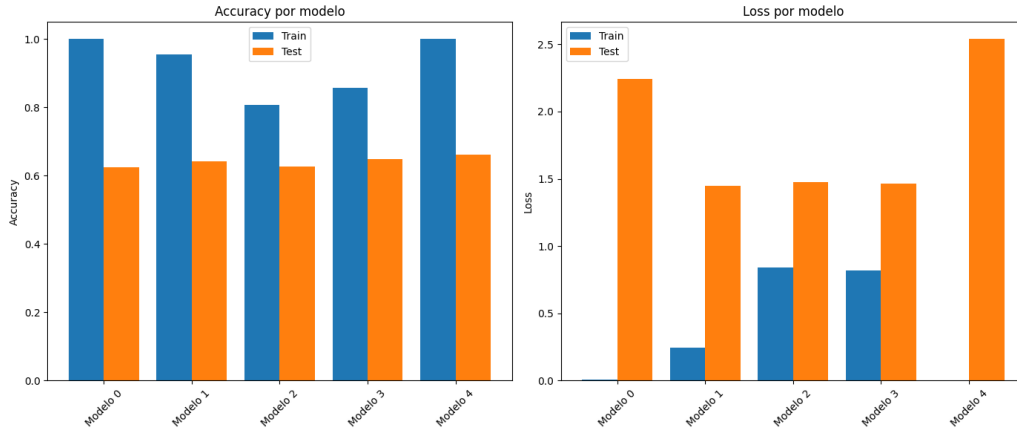


Figura 4: Comparación de desempeño (accuracy y loss) sobre el conjunto de test.

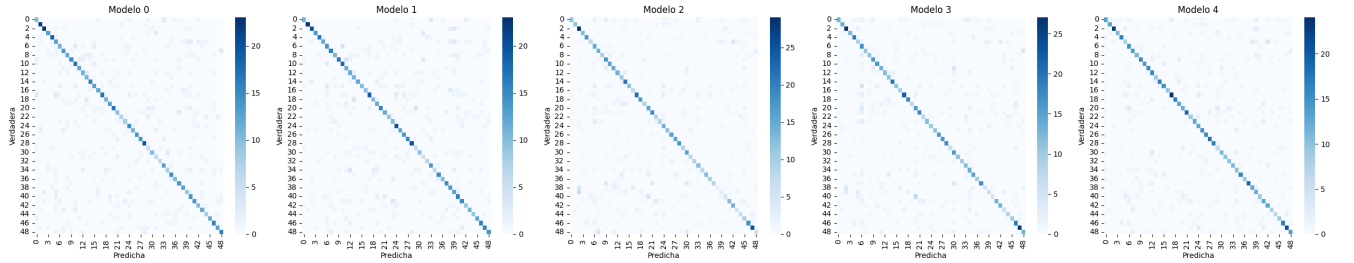


Figura 5: Matrices de confusión de los modelos evaluados.

Se observa que:

- Los modelos M1, M2 y M3 logran un **buen equilibrio entre performance y generalización**. Sus resultados son muy parecidos, lo cual sugiere que la solución óptima no está en arquitecturas más profundas ni en técnicas avanzadas, sino en una buena calibración.
- M0 y M4 muestran un **sobreajuste claro**, cada uno por motivos distintos: M0 por falta de regularización, y M4 por una arquitectura desmedida sin control. Aunque ambos se modelan a la perfección sobre el set de entrenamiento, sus matrices de confusión no son idénticas, lo que indica que no son modelos equivalentes: capturan patrones diferentes pese a su alto sobreajuste.
- Las matrices de M2 y M3 revelan **errores similares en la clasificación**, reforzando la idea de que el dataset impone un límite natural a la performance, más allá de la arquitectura o técnica usada.

En conclusión, el dataset presenta una **complejidad intermedia**: no trivial, pero tampoco exige modelos complejos. Lo clave fue evitar el sobreajuste sin perder capacidad de representación. El mejor enfoque fue una **arquitectura simple con regularización moderada (L2)** y descenso por gradiente clásico. Técnicas como dropout o Adam ayudaron a la estabilidad pero no mejoraron la accuracy final. En resumen, modelos bien calibrados y con pocas capas fueron más efectivos que soluciones sobrecargadas o sobreactualizadas.

Modelo final: M1. Basándonos en todo lo anterior, se eligió el modelo M1 para realizar las predicciones requeridas. Es el modelo con mejor **balance rendimiento-generalización**, presenta buena estabilidad y consistencia en las métricas de validación y test, y es simple, reproducible y fácil de ajustar si fuera necesario. Con este modelo se generaron las probabilidades a posteriori sobre el conjunto `X_COMP.npy`, guardadas en el archivo `Bianchi_Martin_predicciones.csv`.

Apéndice: Enlace al repositorio del proyecto

En el siguiente enlace de Google Drive se encuentran todos los archivos relacionados con el proyecto, incluyendo el notebook explicativo, códigos de los modelos, datos utilizados, un archivo README, los requisitos (`requirements.txt`) y otros recursos necesarios: https://drive.google.com/drive/folders/1CLJh0CKnra2RDH8hm3Vf2KRsGuEbpQwo?usp=drive_link